



Compte rendu de TP 2 :

Java EE



Module : *Applications Web Avancées avec Java*

Filière : *Transformation Digitale & Intelligence Artificielle 2.*

Année : *2025/2026.*

Réalisé par :

- Es-saouiqui Amine.

Supervisée par :

- Dr. Cherradi Mohamed



Table de Matière :

I.	Introduction :	4
II.	Outils et Environnement de Travail :	4
1)	- Environnement de Développement et Exécution :	4
2)	- Gestion de Projet et Construction	5
3)	- Couche Présentation :	5
III.	Architecture du Projet :	5
1)	- Structure des fichiers et enchaînement :	5
2)	- Définition des Entités :	6
3)	- Flux de fonctionnement :	7
IV.	Réalisation Technique :	8
1)-	Organisation de l'arborescence du projet :	8
2)-	Description de l'arborescence :	9
a)	- Le dossier src/main/java :	9
b)	Le dossier src/main/webapp :	9
c)	Le dossier sécurisé /WEB-INF/ :	9
d)	Le fichier pom.xml :	10
3)-	Définition des Entités :	11
a)	. L'entité Client.java :	11
b)	L'entité Reservation :	12
4)-	Définition de la page d'accueil :	12
5)-	Implémentation des Servlets :	13
a)	- La servlet CreerClient.java :	13
b)	- La servlet CreerReservation.java :	14
6)-	Définition des page de Saisies :	15
7)-	Pages de Résultats et Sécurisation:	19
V.	Conclusion :	21



Table de Figures :

Figure 1: Logo de Java	4
Figure 2 : Logo Tomcat.....	4
Figure 3 : Logo de Maven.....	5
Figure 4 : Logo de Bootstrap.....	5
Figure 5 :Logo de Tailwind	5
Figure 6 : Diagramme de classe entre l'entité Réservation et Client	6
Figure 7 : Flux de Fonctionnement de l'application	7
Figure 8 : Page d'accueil	12
Figure 9 : Formulaire de création de Client	17
Figure 10 : Formulaire de création de Réservation	18
Figure 11 : Alert d'erreur	19
Figure 12 :Page infoClient.....	20
Figure 13 : Page infoReservation	20

Table des Scripts :

Script 1 : Contenu de fichier pom.xml dans le projet	11
Script 2 : Servlet CreerClient.java	14
Script 3 : Servlet CreerReservation.java	15
Script 4 : Scriptlet exécute un message d'erreur si un champs n'était pas rempli	16



L'objectif principal de ce TP est de se familiariser avec les technologies **Servlet** et **JSP**, piliers de la plateforme **Java EE**. À travers la réalisation d'une application de type **Maven**, nous aborderons la manipulation des requêtes HTTP, le traitement des données de formulaires et la génération de réponses dynamiques.

I. Introduction :

Le projet consiste à concevoir un système de gestion basique articulé autour de deux entités majeures :

- **La gestion des Clients** : *permettant l'inscription et l'enregistrement des coordonnées (nom, prénom, téléphone et e-mail).*
- **La gestion des Réservations** : *permettant d'associer un client à une prestation spécifique définie par son type, son prix et sa vue.*

II. Outils et Environnement de Travail :

La réalisation de cette application repose sur un écosystème technique cohérent permettant de structurer le code, de gérer les dépendances et d'assurer une exécution fluide du serveur web.

1) - Environnement de Développement et Exécution :

- **Java** : *Langage de programmation principal utilisé pour la logique métier et la création des Servlets.*
- **Java EE** : *Plateforme de référence pour le développement d'applications d'entreprise, utilisée ici pour ses composants Servlet et JSP.*



Figure 1: Logo de Java

- **Apache Tomcat v9+** : *Serveur d'applications indispensable agissant comme conteneur de servlets. Il assure la réception des requêtes HTTP et le rendu des pages JSP, généralement configuré sur le port 8080 comme illustré dans le sujet.*



Figure 2 : Logo Tomcat

2) - Gestion de Projet et Construction

- **Apache Maven** : *Outil de gestion de projet utilisé pour l'automatisation de la compilation et la gestion des dépendances (via le fichier pom.xml). Il permet d'inclure proprement les Servlets nécessaires au projet.*



Figure 3 : Logo de Maven

3)– Couche Présentation :

Pour concevoir des interfaces conformes aux captures d'écran fournies, les outils suivants ont été intégrés:

- **Bootstrap** : *Utilisé pour la structure globale (grilles, boutons "Ajouter").*
- **Tailwind CSS** : *Employé pour un style moderne et rapide.*

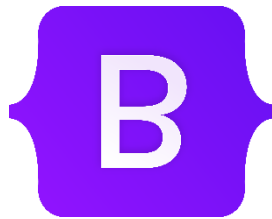


Figure 4 : Logo de Bootstrap

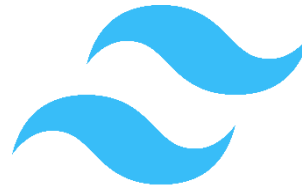


Figure 5 :Logo de Tailwind

III. Architecture du Projet :

L'architecture de l'application est basée sur un flux de données direct entre les interfaces utilisateur et les scripts de traitement Java, comme illustré dans le schéma technique du TP.

Elle s'articule autour des composants suivants :

1) - Structure des fichiers et enchaînement :

Le projet suit une logique de navigation précise où chaque action de l'utilisateur déclenche un cycle de traitement:

- Les pages de saisie (JSP) : **Inscription.jsp** et **Reservation.jsp** servent d'interfaces pour collecter les informations du client.
- Les scripts de traitement (Servlets) : Les fichiers **creerClient.java** et **creerReservation.java** reçoivent les données, les manipulent et décident de la suite du flux.



- Les pages de résultat (JSP) : **infoClient.jsp** et **infoReservation.jsp** affichent les données finales une fois la validation terminée.



Il faut bien mentionner que les pages de résultat sont des fichiers privés. Accessibles juste via les **Servlets**, non directement par le navigateur .

2)- Définition des Entités :

L'application manipule deux types de structures de données:

- L'entité Client : Regroupe le nom, le prénom, le numéro de téléphone et l'adresse e-mail.
- L'entité Réservation : Elle contient l'objet **Client** complet, auquel s'ajoutent le type de chambre, le prix et la vue.

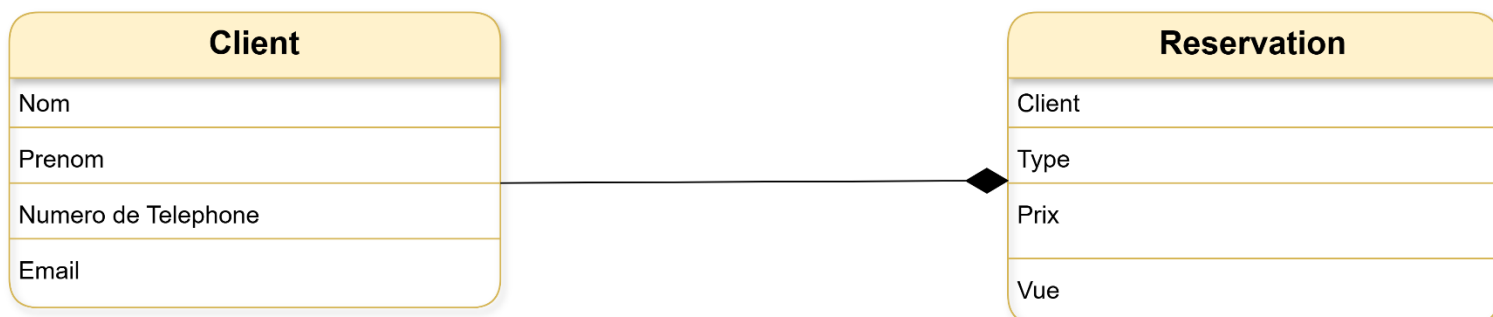


Figure 6 : Diagramme de classe entre l'entité Réservation et Client



3)- Flux de fonctionnement :

Conformément au travail demandé:

- L'utilisateur interagit avec le client (navigateur) pour remplir un formulaire (Soit Client ou Réservation) .
- La requête est transmise à la Servlet correspondante qui extrait les paramètres.
- En cas de succès, les données sont transmises à la page de confirmation (*privée*) pour un affichage structuré.

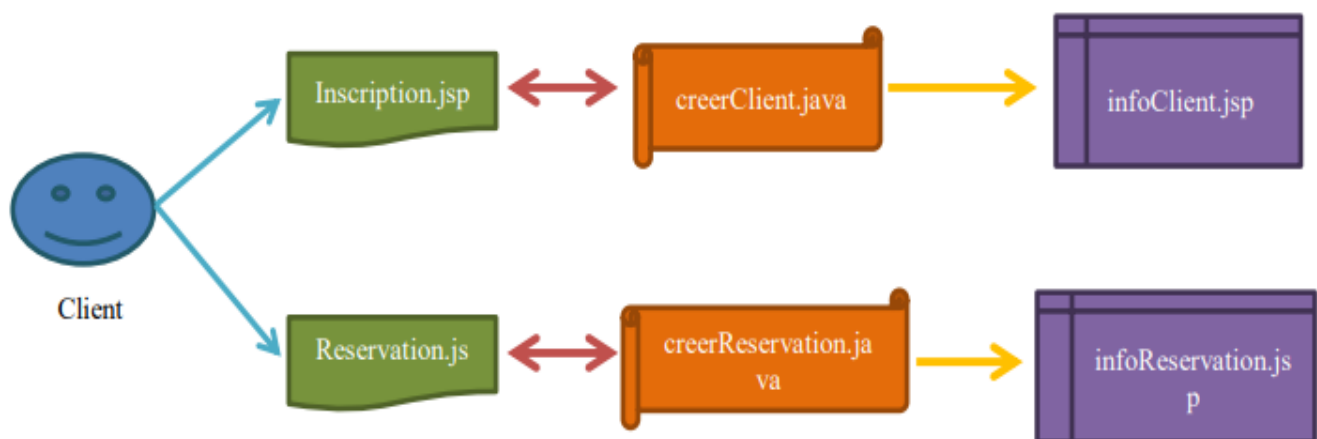


Figure 7 : Flux de Fonctionnement de l'application



1)- Organisation de l'arborescence du projet :

L'organisation des fichiers sous l'environnement de développement respecte la structure standard d'un projet Maven, segmentant le code source des ressources web:

```

Tp2\
|---pom.xml
|
+---lib\
|
+---src\
|   +---main\
|       +---java\
|           +---com\
|               +---example\
|                   +---models\
|                       Client.java
|                       Reservation.java
|                   +---servlets\
|                       CreerClient.java
|                       CreerReservation.java
|       +---webapp\
|           index.jsp
|       +---views\
|           inscription.jsp
|           reservation.jsp
|       \---WEB-INF\
|           infoClient.jsp
|           infoReservation.jsp
|           web.xml

```

Arborescence 1 : Arborescence du Projet



2)- Description de l'arborescence :

L'organisation adoptée suit les standards de développement professionnel, séparant la logique métier des interfaces de présentation.

a) - Le dossier `src/main/java` :

C'est le cœur logique de l'application. Le code est divisé en packages pour respecter la séparation des responsabilités :

- **com.example.models** : Ce package contient les classes POJO ([Client.java](#) et [Reservation.java](#)). Elles définissent les structures de données (nom, prénom, prix, etc.) manipulées entre les différentes pages.
- **com.example.servlets** : Ce package regroupe les contrôleurs [CreerClient.java](#) et [CreerReservation.java](#) . Ce sont ces classes qui reçoivent les données envoyées par les formulaires via le protocole HTTP et orchestrent la réponse du serveur.

b) Le dossier `src/main/webapp` :

Ce dossier contient tout ce qui est envoyé au navigateur de l'utilisateur :

- **index.jsp** : Il s'agit de la page d'accueil de l'application (le point d'entrée `localhost:8080/tp2/`).
- **views/** : Ce sous-dossier contient les formulaires de saisie ([inscription.jsp](#) et [reservation.jsp](#)). Ils sont placés ici car ils doivent être accessibles directement par le client pour initier une action.

c) Le dossier sécurisé `/WEB-INF/` :

C'est un dossier spécial dont le contenu est invisible pour l'utilisateur final depuis son navigateur :

- **infoClient.jsp et infoReservation.jsp** : Ces pages affichent les résultats après validation. En les plaçant dans `/WEB-INF/`, on s'assure que l'utilisateur ne peut pas les consulter "à vide" sans être passé par le traitement de la [Servlet](#) correspondante.
- **web.xml** : C'est le descripteur de déploiement. Il configure le serveur [Tomcat](#) en faisant le lien entre les URLs tapées par l'utilisateur et les classes Java physiques.



d) Le fichier pom.xml :

Fichier de configuration de Maven, il automatise la gestion des bibliothèques nécessaires (Java EE API) et facilite la compilation du projet en un fichier `.war` prêt à être déployé sur **Tomcat**.



Pour assurer la compilation et le déploiement correct de l'application sur le serveur Tomcat, le fichier de configuration Maven pom.xml doit impérativement définir les paramètres suivants :

- **Type de Packaging** : Défini sur `<packaging>war</packaging>` pour générer une archive web déployable.
- Voici un exemple de configuration fichier `pom.xml` :

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>tp2</artifactId>
  <!-- Définir le type de packaging au format war -->
  <packaging>war</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>tp2 Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
    <!-- Specifier l'API Servlet Javax Servlet (Optionnel vue qu'il pre-existe
sur Tomcat ) -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
```



```
<scope>provided</scope>
</dependency>

</dependencies>
<build>
  <finalName>tp2</finalName>
</build>
<properties>
  <!-- Specifier la version JVM a utiliser pour la compilation -->
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>
</project>
```

Script 1 : Contenu de fichier pom.xml dans le projet

3)- Définition des Entités :

Avant de traiter les requêtes, nous avons défini deux classes Java simples (POJO) qui servent de conteneurs pour les données de notre application.

a) . L'entité *Client.java* :

Cette classe représente l'utilisateur du service. Elle possède des attributs privés, un constructeur complet et des *getters/setters* pour permettre l'accès aux données.

• Attributs :

- String nom
- String prenom
- String telephone
- String email



b) L'entité Reservation :

Cette entité est plus complexe car elle illustre une relation de composition. Une réservation ne peut pas exister sans un client associé.

- **Attributs :**

- Client client
- String type
- double prix
- String vue

4)- Définition de la page d'accueil :

La page d'accueil est présentée par le fichier `index.jsp`. Elle accueille l'utilisateur avec un message et propose des liens de navigation vers les différents services : [Inscription](#) et [Reservation](#) .

L'aperçu de la page d'accueil dans le navigateur :

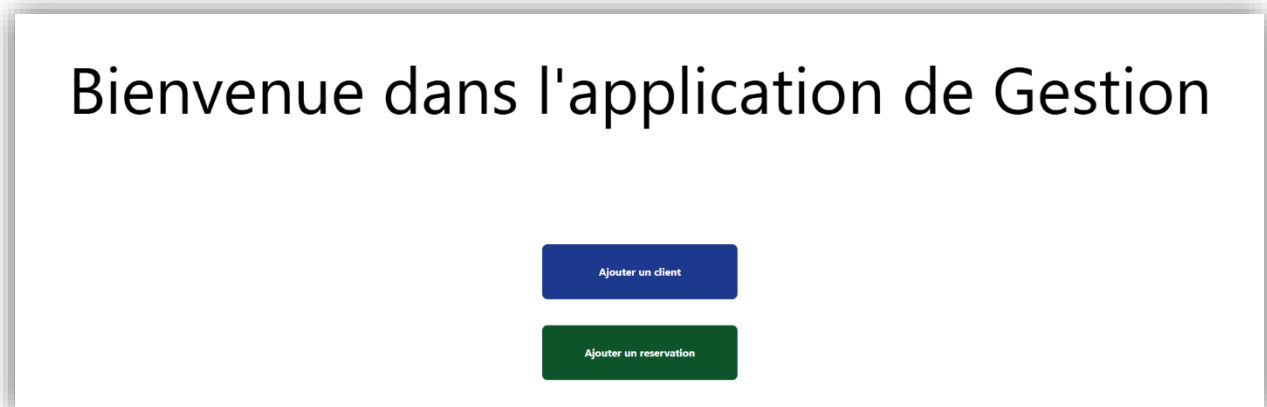


Figure 8 : Page d'accueil



5) -Implémentation des Servlets :

a) – La servlet `CreerClient.java` :

La Servlet `CreerClient` est le contrôleur principal pour la gestion des utilisateurs. Elle assure la liaison entre le formulaire de saisie et la page de confirmation grâce à un mécanisme de mapping et de validation.

- Au lieu d'utiliser le fichier `web.xml`, nous avons utilisé l'annotation moderne `@WebServlet("/creerClient")` qui déclare la classe comme une Servlet et définit son URL de mapping présenté par avec le chemin `/creerClient`.
- La classe `CreerClient` doit évidemment hériter de la classe `HttpServlet`.
- En tenant compte que le formulaire est envoyée en appelant la méthode **POST**. Nous avons défini la logique de création du client en utilisant la méthode `doPost()`. Il permet d'exécuter le cycle de vie suivante :
 1. **Extraction** : Elle récupère les paramètres `nom`, `prenom`, `telephone` et `email` envoyés par la requête HTTP.
 2. **Contrôle de saisie** : Une structure conditionnelle vérifie que les champs ne sont pas vides :
 - **Si valide** : Un objet `Client` est instancié et stocké dans l'objet `req` via `setAttribute("client", client)`. Le flux est ensuite redirigé vers la vue sécurisée `/WEB-INF/infoClient.jsp` via un `RequestDispatcher`.
 - **Si invalide** : La Servlet renvoie l'utilisateur vers le formulaire initial en activant un indicateur d'erreur, ce qui permet d'afficher le message d'erreur.

Voici la définition de la servlet `CreerClient` :



```
package com.example.servlets;

import java.io.IOException;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.example.models.Client;

@WebServlet("/creerClient")
public class CreerClient extends HttpServlet{

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
        String nom = req.getParameter("nom");
        String prenom = req.getParameter("prenom");
        String tel = req.getParameter("telephone");
        String email = req.getParameter("email");
        // Verifier est-ce que tous les champs sont bien remplis :
        if(!nom.trim().isEmpty() && !prenom.trim().isEmpty() && !tel.trim().isEmpty() &&
!email.trim().isEmpty()){
            // Creer un client
            Client client = new Client(email, nom, prenom, tel);
            req.setAttribute("client",client);
            RequestDispatcher rd = req.getRequestDispatcher("/WEB-INF/infoClient.jsp");
            rd.forward(req, resp);
        }else{
            RequestDispatcher rd = req.getRequestDispatcher("views/inscription.jsp");
            req.setAttribute("error", true);
            rd.forward(req, resp);
        }
    }
}
```

Script 2 : Servlet CreerClient.java

b) – La servlet CreerReservation.java :

La Servlet **CreerReservation** suit une architecture identique à celle de **CreerClient** (récupération, validation et redirection).

En cas de succès, l'objet de la classe **Reservation** complet est transmis à la vue sécurisée **/WEB-INF/infoReservation.jsp** .

En cas d'erreur, le mécanisme de retour au formulaire avec l'alerte "Ooops, erreur !!!" est déclenché, garantissant ainsi une expérience utilisateur cohérente sur l'ensemble de l'application.



```

package com.example.servlets;

import java.io.IOException;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.example.models.Client;
import com.example.models.Reservation;

@WebServlet("/creerReservation")
public class CreerReservation extends HttpServlet{

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
        String nom = req.getParameter("nom");
        String prenom = req.getParameter("prenom");
        String tel = req.getParameter("telephone");
        String email = req.getParameter("email");
        String type = req.getParameter("type");
        double prix = Double.parseDouble(req.getParameter("prix"));
        if(!nom.trim().isEmpty() && !prenom.trim().isEmpty() && !tel.trim().isEmpty() &&
!email.trim().isEmpty() && !type.trim().isEmpty() && prix != 0.0 ){
            Client client = new Client(email, nom, prenom, tel);
            Reservation reservation = new Reservation(client, prix, type);
            req.setAttribute("reservation", reservation);
            RequestDispatcher dispatcher = req.getRequestDispatcher("/WEB-
INF/infoReservation.jsp");
            dispatcher.forward(req, resp);
        }else {
            RequestDispatcher rd = req.getRequestDispatcher("views/reservation.jsp");
            req.setAttribute("error", true);
            rd.forward(req, resp);
        }
    }
}

```

Script 3 : Servlet CreerReservation.java

6)- Définition des page de Saisies :

- Les pages de saisie ([inscription.jsp](#) et [reservation.jsp](#)) constituent l'interface directe avec l'utilisateur. Elles sont conçues pour collecter les données nécessaires avant leur traitement par les Servlets.
- Les formulaires utilisent la méthode **POST** pour garantir la confidentialité des données transmises.
- L'attribut action pointe vers les mappings correspondant aux Servlets ([/creerClient](#) et [/creerReservation](#)).



En cas de l'erreur le **scriptlet** suivant dessus sera exécuté :

```
<!-- Verifier si un erreur est apparue -->

<%
    Boolean isError = (Boolean) request.getAttribute("error");
    if(isError!=null && isError == true){
%>
    <h3 class="text text-danger text-center">
        <%= "Oops,erreur !!! Vous devez remplir tous les champs" %>
    </h3>
<% } %>
```

Script 4 : Scriptlet exécute un message d'erreur si un champs n'était pas rempli

- L'attribut action des formulaires utilise la méthode **request.getContextPath()**. Cette approche garantit la portabilité de l'application.

Voici un exemple d'utilisation :

```
action= <%= request.getContextPath() + "/creerReservation" %> #l'url doit etre :
/tp2/creerReservation
```


Voici l'aperçu des deux pages :



Ajouter un client :

Espace Client

Nom :

Prenom:

Telephone:

Email :

Ajouter Client

Application de gestion de client

Figure 9 : Formulaire de création de Client

Ajouter une Reservation :

Espace Reservation

Nom :

Prenom:

telephone:

email :

Type :

Simple

Simple

Double

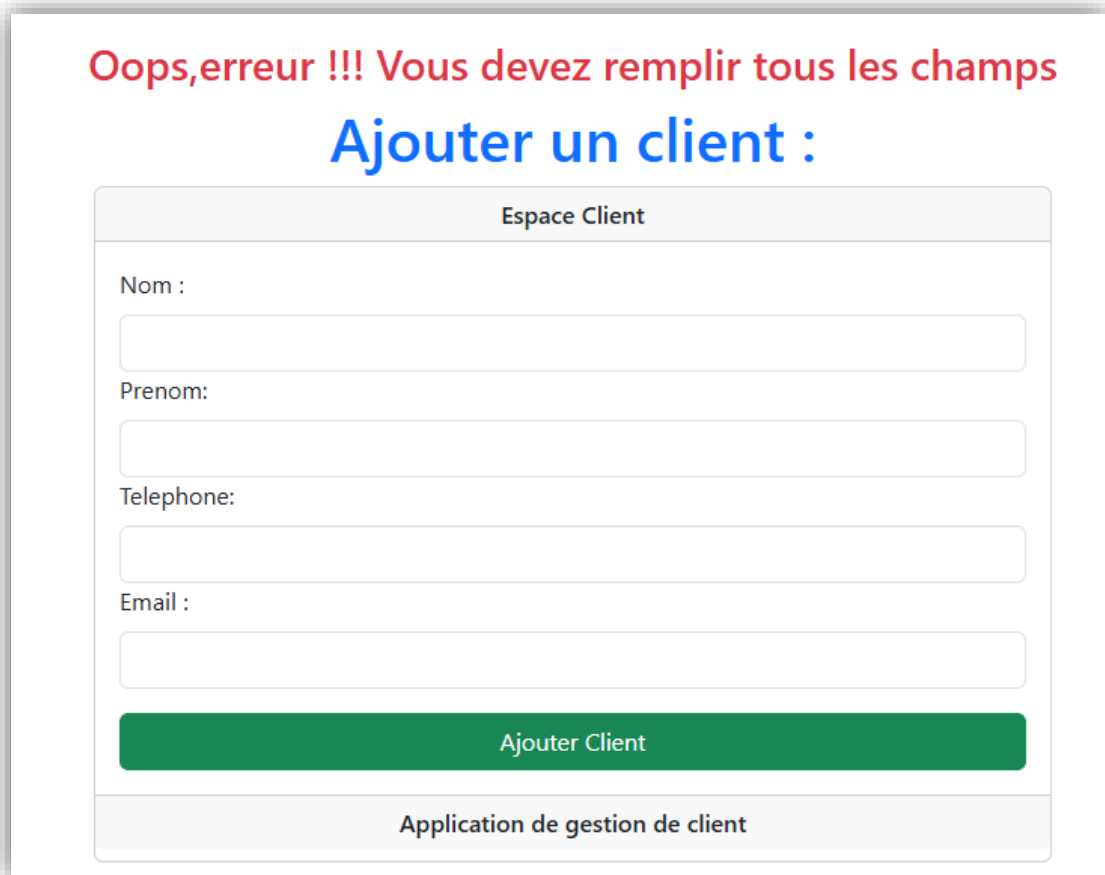
Triple

Ajouter Reservation

Application de gestion de reservation

Figure 10 : Formulaire de création de Réservation

En cas de l'erreur :



The screenshot displays a web application interface. At the top, a red error message reads "Oops, erreur !!! Vous devez remplir tous les champs". Below this, the title "Ajouter un client :" is shown in blue. The main form, titled "Espace Client", contains four input fields labeled "Nom :", "Prenom:", "Telephone:", and "Email :", each followed by an empty text box. A green button labeled "Ajouter Client" is positioned below the form. At the bottom of the page, a footer reads "Application de gestion de client".

Figure 11 : Alert d'erreur

7)- Pages de Résultats et Sécurisation:

Les pages de succès ([infoClient.jsp](#) et [infoReservation.jsp](#)) ont été placées dans le répertoire **/WEB-INF/**. Ce choix architectural garantit que l'affichage des données sensibles ne peut se faire qu'après un traitement valide par une Servlet.

Contrairement aux formulaires, ces pages ne reçoivent pas de paramètres bruts, mais des objets **Java complets**. Nous utilisons les expressions JSP pour extraire les données.



À l'inverse des objets **implicites**, les objets issus de classes personnalisées doivent être importés dans la page JSP via la directive `<%@ page import="..." %>`. Une fois importés, ils peuvent être manipulés dans des scriptlets ou récupérés depuis la requête via `request.getAttribute()`.

Voici l'aperçu des pages `infoClient.jsp` et `infoReservation.jsp` :

Client Ajoute avec succes

Affichage des donnee client :

Nom :	Prenom :	telephone :	email :
Es-saouioui	Amine	07149213154	amineessa02@gmail.com

[Retourne au formulaire d'ajout](#)

Figure 12 :Page infoClient

Reservation Ajoute avec succes

Affichage des donnee Reservation :

Nom :	Prenom :	telephone :	email :	prix :	type :
Es-saouioui	Amine	04652101236	amineessa02@gmail.com	100.0	Simple

[Retourne au formulaire d'ajout](#)

Figure 13 : Page infoReservation



Ce TP a permis de mettre en place une gestion complète de réservations hôtelières via des Servlets et des JSP. L'application assure la validation des données en amont, bloquant toute saisie incomplète par l'affichage de l'alerte.

V. Conclusion :

Il garantit la protection des vues sensibles , l'architecture garantit un flux de données sécurisé entre la récupération des paramètres et l'affichage final des objets Client et Reservation.